AI Security Findings

Overall risk: CRITICAL | Total findings: 8

Executive summary

Risk overview

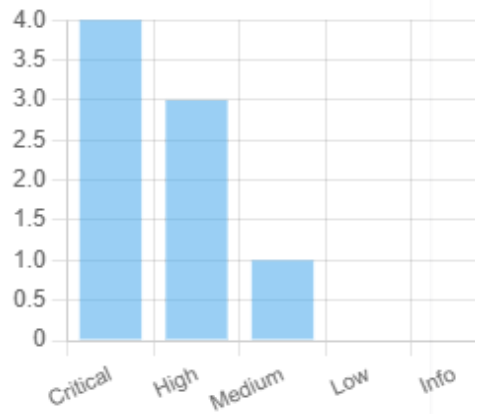
Total findings: 8. Critical: 4, High: 3, Medium: 1, Low: 0.

Technical summary

[
{
"title": "Unsafe pickle deserialization from untrusted URL",
"severity": "critical",
"location": "load_model_from_url function, line 18",
"description": "Downloads and deserializes pickle data from untrusted URLs without validation, enabling remote code execution via crafted payloads.",
"recommendation": "Avoid pickle deserialization from untrusted sources. Use safer formats (e.g., JSON) or sandbox deserialization.",
"source_file": "C:\\Users\\conve\\AppData\\Local\\Temp\\tmpz1qaf4u_\\owasp_llm03_supply_chain.py",
"owasp_category": "A03:2017 - Sensitive Data Exposure"
},
{
"title": "Unsafe pickle deserialization from user-supplied file path",
"severity": "critical",
"location": "load_local_model function, line 28",
"description": "Loads pickle files from user-supplied paths without validation, risking path traversal and remote code execution via malicious pickle files.",
"recommendation": "Validate and restrict file paths. Avoid loading untrusted pickle files; use safer serialization and sandboxing.",
"source_file": "C:\\Users\\conve\\AppData\\Local\\Temp\\tmpz1qaf4u_\\owasp_llm03_supply_chain.py",
"owasp_category": "A03:2017 - Sensitive Data Exposure"
},
{
"title": "Unsanitized subprocess.run() with shell=True allows command injection",
"severity": "critical",
"location": "install_model_dependencies function, line 55",
"description": "Uses subprocess.run() with shell=True and unsanitized user input, enabling command injection attacks.",
"recommendation": "Avoid shell=True with unsanitized input. Use argument lists and sanitize or validate inputs rigorously.",
"source_file": "C:\\Users\\conve\\AppData\\Local\\Temp\\tmpz1qaf4u_\\owasp_llm03_supply_chain.py",
"owasp_category": "A01:2017 - Injection"
},
{
"title": "torch.load called with weights_only=False allows arbitrary code execution",
"severity": "critical",
"location": "load_fine_tuned_weights function, line 98",

"description": "torch.load is invoked without restricting deserialization, allowing arbitrary code execution through malicious model files.",
"recommendation": "Use torch.load with weights_only=True or validate and sandbox loaded files to prevent code execution.",
"source_file": "C:\\Users\\conve\\AppData\\Local\\Temp\\tmpz1qaf4u_\\owasp_llm03_supply_chain.py",
"owasp_category": "A01:2017 - Injection"
},
{
"

Severity distribution



Severity	Title	File / Location	Description	Recommendation
CRITICAL	Unsafe pickle deserialization from untrusted URL	`load_model_from_url function, line 18`	The function downloads content from an untrusted URL and deserializes it using pickle.loads() without validation. This allows remote code execution via a crafted pickle payload.	Avoid untrusted pickle deserialization. Use a safe serialization format like JSON, or verify and sandbox the deserialized data before loading.
CRITICAL	Unsafe pickle deserialization from user-supplied file path	`load_local_model function, line 28`	The function loads a pickle file from a user-supplied path without path validation, enabling path traversal and remote code execution through	Validate and restrict file paths. Avoid loading pickle files from untrusted inputs. Prefer safe serialization methods and sandbox deserialization.

Severity	Title	File / Location	Description	Recommendation
			malicious pickle payloads.	
HIGH	Direct GPU memory access using ctypes causes cross-user data leakage	`read_gpu_memory_via_library function, line 41`	The function uses ctypes to load the CUDA driver library and reads GPU memory directly, which can leak sensitive data from other users sharing the same GPU environment.	Avoid direct GPU memory access in shared environments. Use proper GPU APIs with enforced access controls or isolate GPU usage per user.
CRITICAL	Unsanitized subprocess.run() with shell=True allows command injection	`install_model_dependencies function, line 55`	The function uses subprocess.run() with shell=True and injects unsanitized user input directly into the command string, enabling command injection attacks.	Avoid using shell=True with unsanitized input. Use argument lists with subprocess.run and sanitize or validate all inputs.
HIGH	Dynamic import of unverified plugin without signature verification	`load_plugin function, line 61`	The function dynamically imports a plugin from a user-supplied file path without signature verification or sandboxing, enabling arbitrary code execution.	Implement signature checks and sandboxing before dynamically importing untrusted plugins.
MEDIUM	Unpinned dependency versions in requirements allow version substitution attacks	`REQUIREMENTS variable, line 77`	The requirements specify dependencies without exact version pins (e.g., 'openai>=1.0'), which may lead to supply chain attacks using malicious or compromised newer versions.	Pin package versions exactly (e.g., 'openai==1.23.0') to prevent unintended upgrades and ensure integrity.

Severity	Title	File / Location	Description	Recommendation
HIGH	Fetching model config from external URL without integrity checks	`fetch_model_config function, line 85`	The function fetches a model configuration JSON from an external URL without verifying integrity via SHA256 or signatures, risking supply chain injection.	Validate the integrity of downloaded configuration files using cryptographic hashes or signatures before use.
CRITICAL	torch.load called with weights_only=False allows arbitrary code execution	`load_fine_tuned_weights function, line 98`	torch.load is called with weights_only=False (default), which allows deserialization of arbitrary Python objects, enabling remote code execution via malicious model files.	Use torch.load with weights_only=True to avoid arbitrary code execution, or implement proper file validation and sandboxing.